# EXPLOITS UNIVERSITY

**BSc in Information Communication and Technology — Year Three**

**BICT 3202: Object-Oriented Analysis and Design**

**END-OF-SEMESTER EXAMINATION — PAPER B**

| Examination details | |
|---|---|
| Duration | 3 hours (proposed; subject to Faculty approval) |
| Total marks | 100 |
| Examination weighting | 60% of the overall module assessment, as stated in the course outline |
| Date | _____ |
| Candidate name / number | _____ |

## Instructions to candidates

1. Answer **Question 1 (compulsory)** and **any THREE** questions from Questions 2–5.
2. Question 1 is worth 40 marks. Each question in Section B is worth 20 marks. Your attempted answers must therefore total 100 marks.
3. Answer every subpart of each question you choose. Do not answer more than three questions in Section B.
4. Draw UML diagrams neatly and use standard notation. Label actors, system boundaries, classes, relationships, multiplicities, messages, guards and states where appropriate.
5. State any assumptions you make. A different model can earn full credit when it is logically justified and consistent with the scenario.
6. Marks are awarded for sound analysis and clear, consistent modelling—not artistic presentation.

## Section A — Compulsory integrated case study

### Question 1 — Campus Equipment Lending System (40 marks)

A college resource centre lends laptops, cameras and projectors to students and staff. At present, staff use paper cards to track loans. Records are sometimes incomplete, the same physical copy is promised to more than one person, and overdue items are difficult to identify.

The college plans an equipment-lending system with the following rules:

- Members can search a catalogue and request equipment. Each catalogue item may have several physical copies.
- A librarian approves and records a checkout. A member may have at most two active loans at a time. Every loan records a checkout date and due date.
- A physical copy can have at most one active loan at a time. A returned copy is made available again unless the librarian records that it is damaged; damaged copies are placed in maintenance. On repair completion, hold the copy for the first waiting reservation if the queue is non-empty; otherwise make it Available. A librarian may retire a copy from the catalogue permanently.
- A member may renew a loan once, provided that no unfulfilled reservation (waiting or held) exists for the catalogue item. If no lendable copy of an item is available, a member may join a reservation queue. Reservations are for catalogue items, not particular copies, and are served in first-in, first-out order; a returned copy is held for the first waiting member until checkout or cancellation of that reservation by the member or librarian. After cancellation, hold the copy for the next waiting member, or make it Available if the queue is empty. There is no automatic hold expiry in this practice case. If the first waiting member is inactive or already has two active loans, the hold remains but checkout is refused until the member becomes eligible or cancels the reservation.

- The system sends checkout confirmations and due-date reminders through an external email service. A loan becomes Overdue at or after its due time unless already returned. Return is allowed from OnLoan and Overdue. Use server time; renewal extends the due time and increments renewalCount atomically with the reservation check.
- Only active members and authorised librarians may use protected lending functions. Active loans include OnLoan and Overdue loans. Renewal is allowed only from OnLoan, not Overdue; held-but-uncollected reservations remain unfulfilled. Retire a copy only when it is not on loan or held for a reservation.
- Member eligibility/loan-limit checks, copy-status/hold-owner checks, copy allocation/claim and loan persistence must succeed together in one atomic transaction, respecting any reservation queue. On failed checkout, create no Loan and roll back any new copy allocation/claim; preserve any pre-existing queue hold. A Loan is created directly in OnLoan only on successful issue. Allocation/claim is an atomic checkout operation, not creation of a queue Reservation or a separate Reserved state. A queue Reservation becomes fulfilled when its held copy is issued; all other waiting reservations remain in the queue. Send confirmation only after commit.

Answer the following:

**(a)** State the business problem and the business need. Propose **two measurable success criteria** for the new system. **[4 marks]**

**(b)** Write **three functional requirements** and **two measurable non-functional requirements** for the system (3 × 1 mark for functional requirements; 2 × 1.5 marks for non-functional requirements). **[6 marks]**

**(c)** Draw a UML use-case diagram. Show the system boundary and the actors **Member**, **Librarian** and **Email Service**. Include the main lending use cases and use `<<include>>` or `<<extend>>` only where the relationship is justified. **[6 marks]**

**(d)** Draw a domain/design class diagram containing suitable classes for members, catalogue items, physical copies, loans and reservations. Add meaningful attributes and operations, associations and multiplicities. Include a written constraint or UML note for both the two-loan member limit and the one-active-loan-per-copy rule. Add any application/service class needed for your sequence diagram. **[12 marks]**

**(e)** Draw a sequence diagram for **Issue Loan**. Show the librarian approving/recording the checkout, member eligibility and loan-limit checks, selection and atomic allocation of an Available copy or the copy held for this member in the same transaction as member-limit checks and loan persistence, and confirmation through the email service only after commit. Include an alternative path when no copy is available or the member is ineligible. **[8 marks]**

**(f)** Draw a state-machine diagram for **Loan**. Show an initial node and meaningful states, event-triggered transitions, overdue and return outcomes, and a renewal guard that permits at most one renewal, only from OnLoan and only when no reservation is waiting. **[4 marks]**

---

## Section B — Answer any THREE questions

### Question 2 — Object modelling and object-oriented concepts (20 marks)

**(a)** Define **encapsulation** and **abstraction**, giving one suitable example of each. Briefly distinguish a **class** from an **object**. **[4 marks]**

**(b)** Explain the difference between **generalisation** and **inheritance**. Give a justified class hierarchy and use it to show how polymorphism could allow two specialised objects to respond differently to the same operation. State any policy assumption you introduce; do not contradict the lending rules in the case. **[6 marks]**

**(c)** Explain `<<include>>` and `<<extend>>` in use-case modelling. State which one represents required reused behaviour and which one represents conditional or optional behaviour. Give one valid example of each for the lending system. **[4 marks]**

**(d)** Compare **association**, **aggregation** and **composition**. Give one example of each and explain the meaning or lifecycle assumption behind your choice. **[6 marks]**

## Question 3 — Dynamic and behavioural modelling (20 marks)

A physical copy is **Available** until issued. A returned undamaged copy becomes available again unless held for the first waiting reservation; a damaged copy enters maintenance and cannot be issued until repaired; repair completion respects any waiting reservation queue. A librarian may retire a copy, after which it cannot be lent. The Loan has its own lifecycle: for example, it may become **Overdue** even though the Copy remains **OnLoan**.

**(a)** Define **state**, **event** and **operation**, and distinguish them using one Copy or Loan example. **[4 marks]**

**(b)** Draw a state-machine diagram for a **Copy**. Include an initial state, suitable states and event-triggered transitions for issue, return, damage, repair and retirement. Show a suitable issue guard and do not model **Overdue** as a Copy state. **[8 marks]**

**(c)** Compare a **sequence diagram**, a **communication diagram** and an **activity diagram**. For each, state the question it answers and one situation in which it is useful. **[4 marks]**

**(d)** Explain how a guard and an atomic checkout operation can prevent two concurrent requests from receiving the same physical copy. Identify the invariant that must remain true. **[4 marks]**

## Question 4 — Unified Software Development Process (20 marks)

The college wants a first release of the lending system for one resource centre, followed by later releases for other campuses. The equipment inventory data is incomplete, and staff need to test the process before the system is rolled out widely.

**(a)** Explain the four Unified Process phases—**Inception, Elaboration, Construction and Transition**—and the primary decision or outcome associated with each. **[8 marks]**

**(b)** Propose a suitable iterative plan for this project. Identify at least two important workflows, one risk to address early, and two artefacts or deliverables that would help the team and stakeholders evaluate progress. **[8 marks]**

**(c)** The team is considering reuse of an existing authentication or notification component. State and explain **two suitability checks** it should make before adopting it, and one possible benefit of reuse. **[4 marks]**

## Question 5 — From analysis to design and architecture (20 marks)

A first draft puts catalogue search, member management, loan eligibility, due-date calculation, database queries, email sending and overdue reports into one `ResourceCentreSystem` class. The user interface also creates database records directly.

**(a)** Distinguish **object-oriented analysis** from **object-oriented design**. Trace the requirement "an eligible member borrows an available copy" through a use case to one plausible design class and operation. **[4 marks]**

**(b)** Diagnose the design in the scenario. Explain its cohesion and coupling problems, propose a more appropriate allocation of responsibilities across classes or services, and discuss one strength and one limitation of the object-oriented approach in this context. **[8 marks]**

**(c)** Propose a layered architecture and component arrangement for the improved system. Show the major layers/components and dependency direction, explain how interfaces support adaptability or reuse, and state two ways a CASE/UML tool could help maintain the design. **[8 marks]**

**END OF PAPER B**